

单位代码： 10293 密 级： 公开

南京邮电大学

专业学位硕士学位论文



论文题目： 边缘计算中面向快速函数启动与低内存开销的
在线容器调度方法

学 号	<u>1025040808</u>
姓 名	<u>王宇杰</u>
导 师	<u>杨教授</u>
专业学位类别	<u>计算机科学</u>
类 型	<u>应用研究</u>
专业（领域）	<u>计算机技术</u>
论文提交日期	<u>二〇二六年六月</u>

南京邮电大学学位论文原创性声明

本人声明所呈交的学位论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得南京邮电大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

本人学位论文及涉及相关资料若有不实，愿意承担一切相关的法律责任。

研究生学号：_____ 研究生签名：_____ 日期：_____

南京邮电大学学位论文使用授权声明

本人承诺所呈交的学位论文不涉及任何国家秘密，本人及导师为本论文的涉密责任并列第一责任人。

本人授权南京邮电大学可以保留并向国家有关部门或机构送交论文的复印件和电子文档；允许论文被查阅和借阅；可以将学位论文的全部或部分内容编入有关数据库进行检索；可以采用影印、缩印或扫描等复制手段保存、汇编本学位论文。本文电子文档的内容和纸质论文的内容相一致。论文的公布（包括刊登）授权南京邮电大学研究生院办理。

非国家秘密类涉密学位论文在解密后适用本授权书。

研究生签名：_____ 导师签名：_____ 日期：_____

摘要

边缘计算通过将计算、存储与网络能力下沉至用户侧，能够降低云端访问延迟并缓解核心网络压力。在无服务器与云边协同场景中，函数实例通常依赖容器启动和镜像加载完成运行环境初始化。由于边缘服务器资源受限，若对所有服务均保留完整容器，会造成较高的内存占用；若完全依赖冷启动，则会引入明显的响应延迟。因此，如何在有限资源下联合优化容器缓存、任务调度与启动开销，是面向在线强化学习应用的重要问题。

本文围绕边缘计算环境中的在线容器调度问题展开研究。首先，构建包含边缘服务器、服务类型、请求到达、完整容器缓存与 Zygote 缓存的系统模型，给出四类请求处理情形，并建立服务质量与缓存成本联合优化目标。其次，将原问题转化为马尔可夫决策过程，设计以服务器-服务类型对为基本单元的状态表示、离散动作空间和奖励函数。再次，采用近端策略优化算法（PPO）进行在线求解，通过动作掩码保证缓存容量和 Zygote 数量约束始终满足。最后，利用仿真实验对所提方法进行评估，并与贪心、随机、零缓存和全量缓存等策略进行比较。实验结果表明，PPO 方法能够在 QoS 与缓存成本之间取得较好平衡，Zygote 缓存和动作掩码对训练稳定性具有积极作用。

关键词：边缘计算；容器调度；强化学习；PPO；Zygote 缓存

Abstract

Edge computing pushes computation, storage and network capabilities close to users, which can reduce cloud access latency and relieve the pressure of backbone networks. In serverless and cloud-edge collaborative systems, function instances usually depend on container startup and image loading to initialize the runtime environment. Since edge servers are resource constrained, keeping full containers for all services leads to high memory cost, while relying on cold start causes significant response delay. Therefore, jointly optimizing container caching, task scheduling and startup overhead under limited resources is an important problem for online reinforcement learning applications.

This thesis studies online container scheduling in edge computing environments. First, a system model is established, including edge servers, service types, request arrivals, full-container caching and Zygote caching. Four request-processing cases are described, and a joint optimization objective of quality of service and caching cost is formulated. Second, the original problem is transformed into a Markov decision process. The state representation, discrete action space and reward function are designed based on server-service pairs. Third, Proximal Policy Optimization (PPO) is adopted for online decision making, and action masks are used to guarantee capacity constraints and Zygote constraints. Finally, simulation experiments are conducted and compared with greedy, random, zero-cache and always-full strategies. Results show that PPO achieves a good tradeoff between QoS and caching cost, and both Zygote caching and action masking contribute to stable training.

Key Words: Edge computing; container scheduling; reinforcement learning; PPO; Zygote caching

目 录

摘要	I
Abstract	II
插图索引	IV
表格索引	V
第一章 绪论	1
1.1 研究背景与意义	1
1.2 研究内容	1
1.3 论文结构	1
第二章 系统模型	2
2.1 边缘计算系统	2
2.2 容器缓存策略	2
2.3 决策变量	2
第三章 问题建模	3
3.1 请求分类模型	3
3.1.1 Case 1: 缓存不足但服务器未满载	3
3.1.2 Case 2: 缓存覆盖全部请求	3
3.1.3 Case 3: 完整容器覆盖全部请求	3
3.1.4 Case 4: 请求超过服务器处理上限	4
3.2 延迟与 QoS 模型	4
3.3 缓存成本与优化目标	4
第四章 基于 PPO 的在线调度算法	6
4.1 MDP 建模	6
4.2 动作掩码机制	6
4.3 网络结构与训练流程	6
第五章 实验与结果分析	8
5.1 实验设置	8
5.2 训练收敛性分析	8
5.3 策略稳定性与多随机种子实验	9
5.4 消融实验与基线比较	10
5.5 QoS-成本权衡与敏感性分析	11
5.6 训练长度对比	12
第六章 结论与展望	13
参考文献	14
附录 A 程序清单	15
附录 B 攻读专业学位硕士期间撰写的论文	16
附录 C 攻读专业学位硕士期间申请的专利	17
附录 D 攻读专业学位硕士期间参加的科研项目	18
致谢	19

插图索引

图 5.1	PPO 学习曲线	8
图 5.2	PPO 训练诊断结果	9
图 5.3	策略稳定性分析	9
图 5.4	多随机种子收敛曲线	10
图 5.5	消融实验学习曲线	10
图 5.6	PPO 与基线方法对比	11
图 5.7	QoS 与缓存成本权衡	11
图 5.8	关键参数敏感性分析	12
图 5.9	不同训练长度对比	12

表格索引

表 2.1	两类缓存方式比较	2
表 4.1	PPO 训练超参数	6
表 4.2	基于 PPO 的在线容器调度训练流程	7
表 5.1	实验参数设置	8
表 5.2	基线方法性能对比示例	11

第一章 绪论

1.1 研究背景与意义

随着移动互联网、物联网和人工智能应用的快速发展,用户侧产生的数据量持续增长。传统云计算模式需要将任务上传至远端数据中心处理,虽然具备较强的计算能力,但在实时交互、在线推理和短函数调用场景中容易受到网络传输延迟影响。边缘计算通过在靠近用户的位置部署轻量级计算节点,使任务能够在本地或近端完成,从而降低端到端时延。

无服务器计算进一步提升了资源使用的弹性。用户只需提交函数逻辑,平台自动完成容器创建、运行环境初始化和资源回收。然而,函数冷启动通常需要拉取镜像、装载运行时并启动容器,该过程在边缘环境中尤其昂贵。为了降低冷启动开销,平台可以预留完整容器或缓存部分运行时状态。前者启动快但占用内存高,后者占用低但仍需快速初始化,因此需要根据请求到达和服务器状态动态调度。

1.2 研究内容

本文根据边缘容器调度问题的数学建模,完成以下工作:

- 建立包含完整容器缓存与 Zygote 缓存的在线调度模型,刻画请求到达、缓存命中、冷启动和服务器容量之间的关系;
- 设计 QoS 与缓存成本联合优化目标,并给出四种请求分类情形下的请求数量计算方法;
- 将调度问题转化为 MDP,采用 PPO 强化学习算法进行在线求解;
- 使用实验图展示训练收敛、策略稳定性、基线比较、消融实验和敏感性分析结果。

1.3 论文结构

全文共分为六章。第一章介绍研究背景与论文结构;第二章给出系统模型;第三章建立优化问题;第四章介绍 PPO 在线调度方法;第五章进行实验评估;第六章总结全文并展望后续工作。

第二章 系统模型

2.1 边缘计算系统

考虑由多台边缘服务器组成的系统，服务器集合记为 $\mathcal{S}$ 。每台服务器 $s \in \mathcal{S}$ 具有计算能力 f_s 和负载上限 F_s 。系统支持 K 种服务类型，服务集合记为 $\mathcal{K}$ 。type- k 服务单次请求的计算负载为 W_k ，在时隙 t 到达服务器 s 的请求数量为 u_{ks}^t 。服务器最大并行处理能力可近似表示为 F_s/f_s 。

2.2 容器缓存策略

为降低函数启动延迟，服务器可采用两类缓存策略。

- **完整容器缓存**：保存已经初始化的完整运行环境，请求到达后可直接处理，内存开销记为 r_k^{full} 。
- **Zygote 缓存**：保存基础镜像层和预加载运行时等部分状态，需要经过快速初始化才能处理请求，内存开销记为 r_k^{zygo} ，且满足 $r_k^{\text{zygo}} < r_k^{\text{full}}$ 。

表 2.1 两类缓存方式比较

缓存方式	启动延迟	内存开销	适用场景
完整容器缓存	低	高	高热度、稳定请求服务
Zygote 缓存	中	低	请求波动或中等热度服务
冷启动	高	无持续缓存	低热度服务

2.3 决策变量

在每个时隙结束时，系统需要决定下一时隙保留的缓存数量。令

$$e_{ks}^t \in \mathbb{Z}_{\geq 0}, \quad q_{ks}^t \in \mathbb{Z}_{\geq 0} \quad (2.1)$$

分别表示服务器 s 上 type- k 完整容器数量和 Zygote 数量。由于 Zygote 是完整容器的补充缓存，本文要求 $q_{ks}^t < e_{ks}^t$ 。

第三章 问题建模

3.1 请求分类模型

令 $N_{ks}^{\text{full},t}$ 、 $N_{ks}^{\text{zygo},t}$ 和 $N_{ks}^{\text{cold},t}$ 分别表示完整缓存命中、Zygote 命中和冷启动请求数量。根据 e_{ks}^t 、 q_{ks}^t 、 u_{ks}^t 与服务器处理上限的关系，请求处理可分为四种情形。

3.1.1 Case 1: 缓存不足但服务器未满载

当

$$e_{ks}^t + q_{ks}^t < u_{ks}^t < \frac{F_s}{f_s} \quad (3.1)$$

时，完整容器和 Zygote 均被使用，剩余请求采用冷启动：

$$N_{ks}^{\text{full},t} = e_{ks}^t, \quad (3.2)$$

$$N_{ks}^{\text{zygo},t} = q_{ks}^t, \quad (3.3)$$

$$N_{ks}^{\text{cold},t} = u_{ks}^t - e_{ks}^t - q_{ks}^t. \quad (3.4)$$

3.1.2 Case 2: 缓存覆盖全部请求

当

$$e_{ks}^t < u_{ks}^t \leq e_{ks}^t + q_{ks}^t \quad (3.5)$$

时，全部请求可由缓存覆盖：

$$N_{ks}^{\text{full},t} = e_{ks}^t, \quad (3.6)$$

$$N_{ks}^{\text{zygo},t} = u_{ks}^t - e_{ks}^t, \quad (3.7)$$

$$N_{ks}^{\text{cold},t} = 0. \quad (3.8)$$

3.1.3 Case 3: 完整容器覆盖全部请求

当 $u_{ks}^t \leq e_{ks}^t$ 时，所有请求均由完整容器处理：

$$N_{ks}^{\text{full},t} = u_{ks}^t, \quad (3.9)$$

$$N_{ks}^{\text{zygo},t} = 0, \quad (3.10)$$

$$N_{ks}^{\text{cold},t} = 0. \quad (3.11)$$

3.1.4 Case 4: 请求超过服务器处理上限

当

$$e_{ks}^t + q_{ks}^t \leq \frac{F_s}{f_s} < u_{ks}^t \quad (3.12)$$

时, 服务器达到处理上限, 部分请求需要排队或拒绝:

$$N_{ks}^{\text{full},t} = e_{ks}^t, \quad (3.13)$$

$$N_{ks}^{\text{zygo},t} = q_{ks}^t, \quad (3.14)$$

$$N_{ks}^{\text{cold},t} = \frac{F_s}{f_s} - e_{ks}^t - q_{ks}^t. \quad (3.15)$$

3.2 延迟与 QoS 模型

三类请求的处理延迟分别为

$$T_{ks}^{\text{full}} = \frac{W_k}{f_s}, \quad (3.16)$$

$$T_{ks}^{\text{zygo}} = d_k + \frac{W_k}{f_s}, \quad (3.17)$$

$$T_{ks}^{\text{cold}} = c_k + \frac{W_k}{f_s}, \quad (3.18)$$

其中 d_k 为 Zygote 初始化延迟, c_k 为冷启动延迟, 且 $c_k > d_k$ 。系统 QoS 损失定义为

$$\text{QoS}^t = \sum_{s \in \mathcal{S}} \sum_{k \in \mathcal{K}} \left(N_{ks}^{\text{full},t} \omega_1 + N_{ks}^{\text{zygo},t} \omega_2 + N_{ks}^{\text{cold},t} \omega_3 \right), \quad (3.19)$$

其中 $\omega_1 < \omega_2 < \omega_3$ 。

3.3 缓存成本与优化目标

令 a_{ks}^t 表示增加完整容器的单位成本, b_{ks}^t 表示增加 Zygote 的单位成本, 且 $b_{ks}^t < a_{ks}^t$ 。成本仅在缓存数量增加时产生:

$$C_s^t = \sum_k \left(a_{ks}^t \max\{e_{ks}^t - e_{ks}^{t-1}, 0\} + b_{ks}^t \max\{q_{ks}^t - q_{ks}^{t-1}, 0\} \right). \quad (3.20)$$

最终优化目标为

$$\min_{e,q} \sum_t \left(\text{QoS}^t + \lambda \sum_{s \in \mathcal{S}} C_s^t \right), \quad (3.21)$$

满足

$$e_{ks}^t + q_{ks}^t \leq \frac{F_s}{f_s}, \quad (3.22)$$

$$q_{ks}^t < e_{ks}^t, \quad (3.23)$$

$$e_{ks}^t, q_{ks}^t \in \mathbb{Z}_{\geq 0}. \quad (3.24)$$

第四章 基于 PPO 的在线调度算法

4.1 MDP 建模

本文将在线容器调度问题建模为 MDP。对每个服务器-服务类型对 (s, k) ，状态向量为

$$\mathbf{o}_{ks}^t = \left[e_{ks}^{t-1}, q_{ks}^{t-1}, u_{ks}^t, f_s, \frac{F_s}{f_s}, W_k, d_k, c_k, a_k, b_k \right] \in \mathbb{R}^{10}. \quad (4.1)$$

动作定义为 $\mathbf{a}_{ks}^t = (e_{ks}^t, q_{ks}^t)$ ，其中 e 和 q 均为离散整数。奖励函数将原始最小化目标转化为最大化形式：

$$r^t = \left(1 - \frac{\text{QoS}^t}{\text{QoS}_{\max}} \right) - \lambda \frac{\sum_s C_s^t}{C_{\max}}. \quad (4.2)$$

4.2 动作掩码机制

为保证动作合法，本文在采样 q 时引入条件掩码：

$$m_q(i) = \begin{cases} 1, & i < e \text{ 或 } i \leq M - e, \\ 0, & \text{其他}, \end{cases} \quad (4.3)$$

其中 $M = F_s/f_s$ 。非法动作对应的 logits 被设为 $-\infty$ ，从而避免采样违反约束的动作。

4.3 网络结构与训练流程

策略网络采用共享 MLP 结构处理每个 (s, k) 对。共享编码器为两层全连接网络，隐藏维度为 256，激活函数为 Tanh。网络包含三个输出头： e 输出头、 q 输出头和值函数输出头。PPO 训练超参数如表 4.1 所示。

表 4.1 PPO 训练超参数

超参数	符号	取值
折扣因子	γ	0.99
GAE 参数	λ_{GAE}	0.95
裁剪范围	ϵ	0.2
学习率	α	3×10^{-4}
熵系数	c_2	0.01
更新轮数	—	10

表 4.2 基于 PPO 的在线容器调度训练流程

步骤	操作说明
1	初始化策略网络参数 θ 与价值网络参数 ϕ 。
2	重置边缘环境，获得初始状态 $\mathbf{o}^0$ 。
3	在每个时隙输入状态，得到 e 和 q 的动作分布。
4	根据动作掩码采样合法动作 (e_{ks}^t, q_{ks}^t) 。
5	执行动作并获得奖励 r^t 与下一状态 $\mathbf{o}^{t+1}$ ，随后存储轨迹数据。
6	使用 GAE 计算优势函数，并采用 PPO-Clip 目标函数更新网络参数。

第五章 实验与结果分析

5.1 实验设置

实验采用仿真环境进行。默认设置下，系统包含 5 台边缘服务器和 8 类服务，请求到达服从泊松分布。评价指标包括平均奖励、QoS 损失、缓存成本、完整容器命中数、Zygote 命中数和冷启动请求数。所有实验图片均来自本论文项目的第五次实验结果。

表 5.1 实验参数设置

参数	取值
服务器数量 S	5
服务类型数量 K	8
Episode 长度	200
默认请求到达率	1.0
QoS-成本权重 λ	0.5
基线方法	Greedy、Random、ZeroCache、AlwaysFull

5.2 训练收敛性分析

图 5.1 展示了 PPO 训练过程中的奖励变化。整体趋势表明，随着训练轮数增加，策略逐渐从随机探索转向稳定缓存决策，平均奖励提升并在后期趋于平稳。

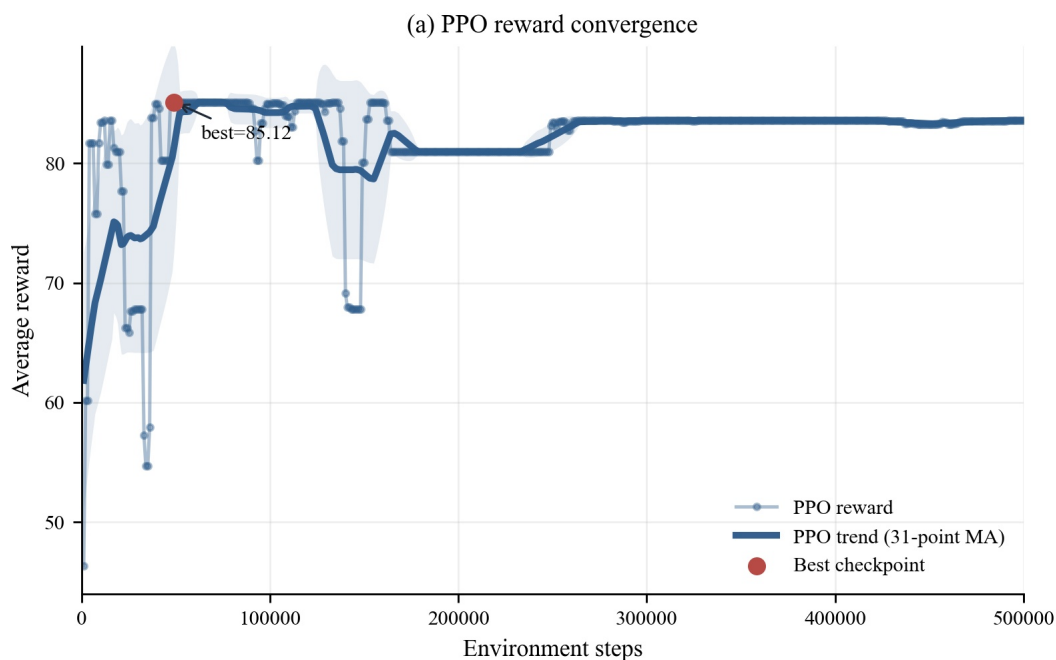


图 5.1 PPO 学习曲线

图 5.2 给出了训练诊断信息，包括策略损失、价值损失和熵等指标。价值损失逐渐下降说明价值函数对状态收益的估计更准确，熵的变化则反映了策略探索强度的调整。

Evaluation metrics during training

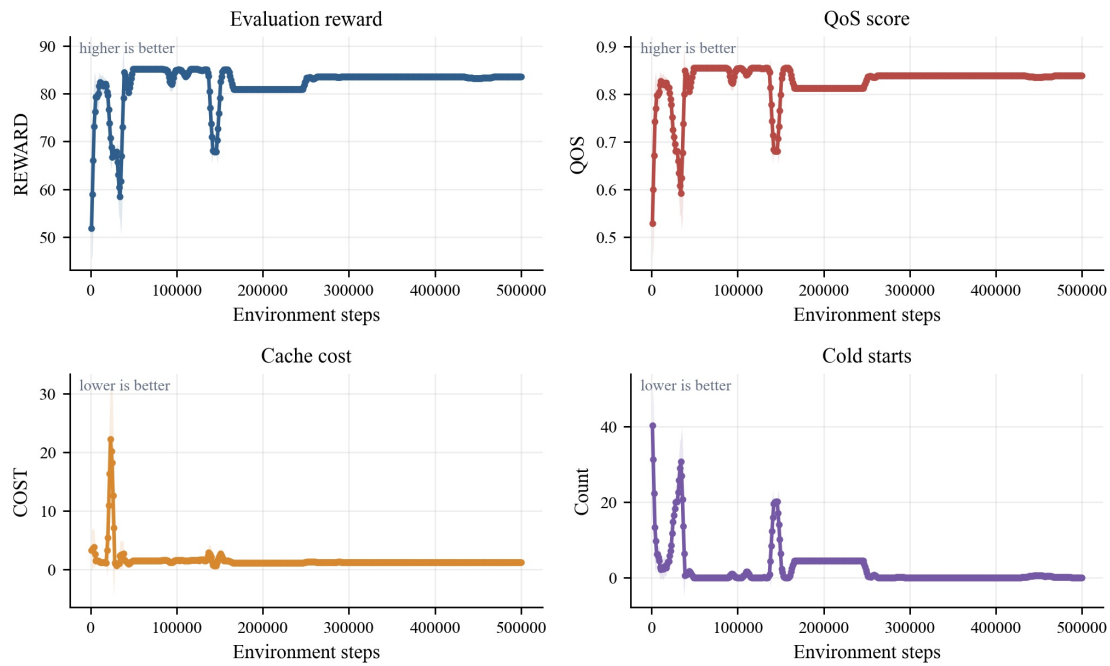


图 5.2 PPO 训练诊断结果

5.3 策略稳定性与多随机种子实验

图 5.3 展示了训练过程中策略稳定性指标。若策略在训练后期变化幅度较小，则说明智能体已学习到较稳定的缓存分配模式。

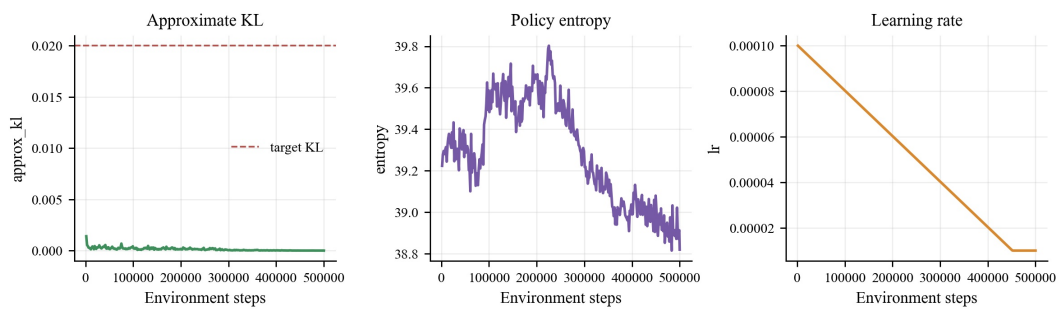


图 5.3 策略稳定性分析

图 5.4 给出了多随机种子条件下的收敛曲线。不同随机种子下曲线趋势基本一致，说明所提方法具有一定鲁棒性。

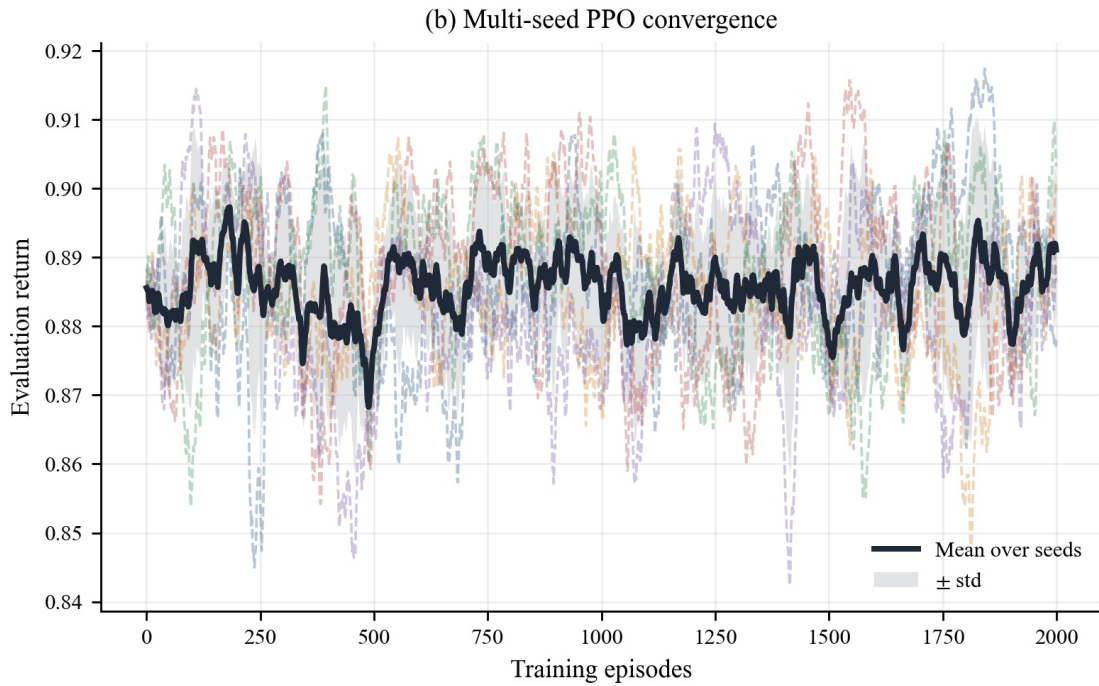


图 5.4 多随机种子收敛曲线

5.4 消融实验与基线比较

图 5.5 展示了不同模型变体的学习曲线。完整 PPO 相比移除 Zygote、移除动作掩码或忽略时序耦合的变体具有更高奖励，说明这些组件均对最终性能有贡献。

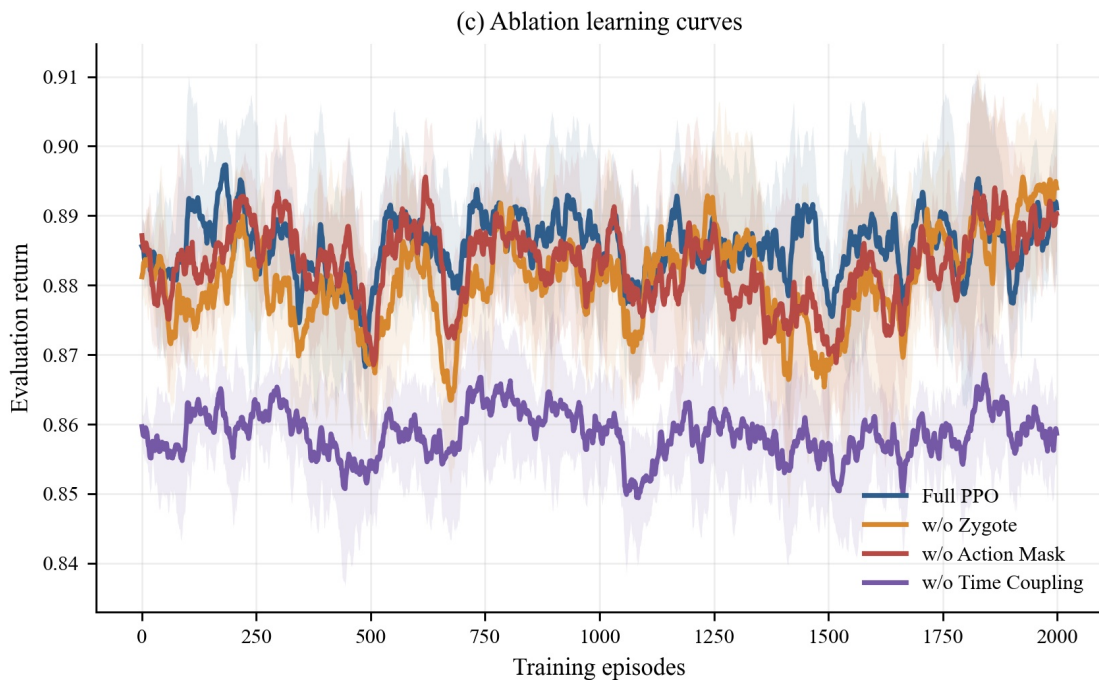


图 5.5 消融实验学习曲线

图 5.6 展示了 PPO 与基线策略的比较。PPO 能够根据请求分布动态调节完整容器和 Zygote 数量，因此在 QoS 与成本之间获得更好的综合收益。

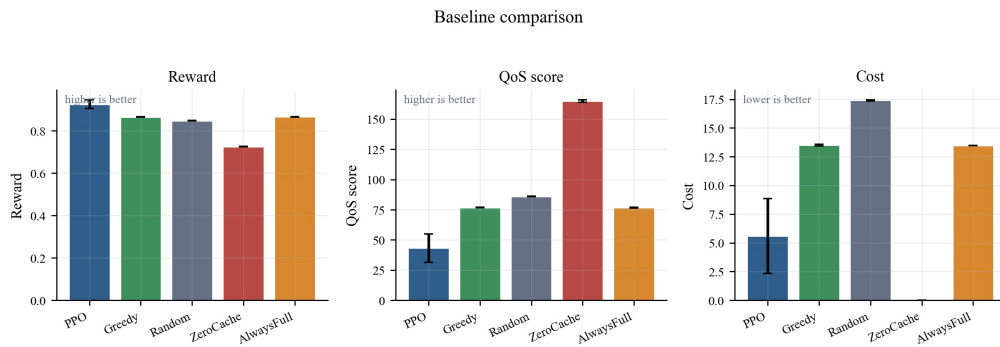


图 5.6 PPO 与基线方法对比

表 5.2 基线方法性能对比示例

方法	Reward $\uparrow$	QoS $\downarrow$	Cost $\downarrow$
PPO	0.925	43.3	5.59
Greedy	0.865	76.8	13.52
Random	0.848	86.0	17.40
ZeroCache	0.725	164.9	0.00
AlwaysFull	0.865	76.6	13.48

5.5 QoS-成本权衡与敏感性分析

图 5.7 展示 QoS 与缓存成本之间的权衡关系。一般而言，更多缓存能够降低冷启动比例并改善 QoS，但会提高缓存维护成本。PPO 通过奖励函数自动学习折中策略。

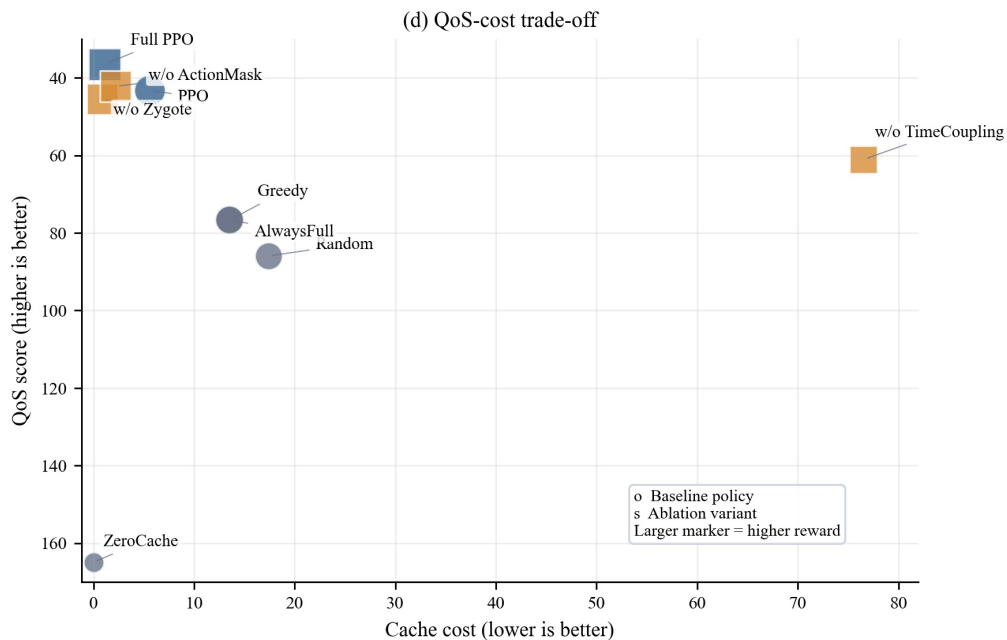


图 5.7 QoS 与缓存成本权衡

图 5.8 展示了关键参数敏感性分析结果，包括权重参数、问题规模和请求到达率等因素。实验结果表明，请求到达率过高时，冷启动比例明显上升，说明系统容量规划仍然重要。

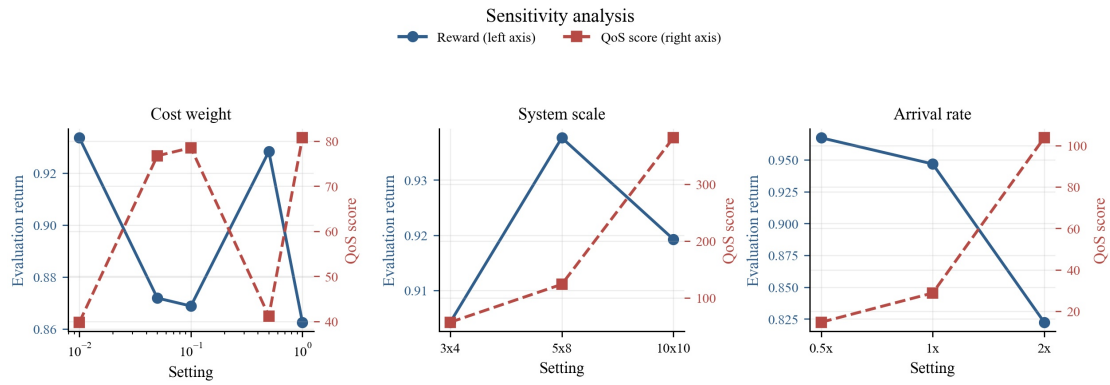


图 5.8 关键参数敏感性分析

5.6 训练长度对比

图 5.9 展示了不同训练长度下策略性能的变化。较长训练通常有助于获得更稳定的策略，但也会带来更高训练成本，因此实际部署时可根据任务规模选择合适的训练轮数。

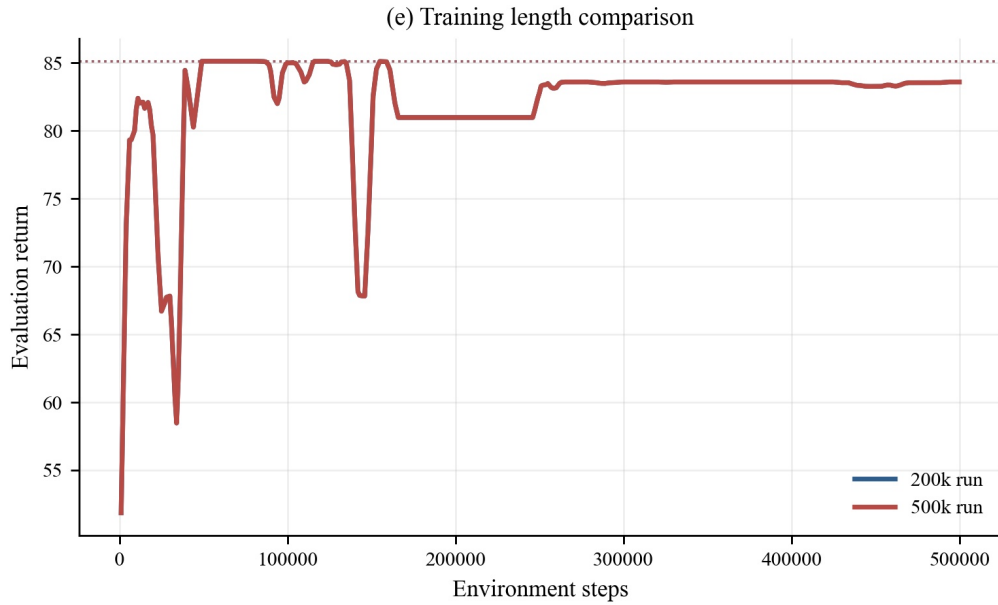


图 5.9 不同训练长度对比

第六章 结论与展望

本文针对边缘计算环境下函数容器启动延迟和缓存资源受限的问题，建立了完整容器缓存与 Zygote 缓存并存的在线调度模型，并采用 PPO 强化学习算法进行求解。通过状态建模、奖励函数设计和动作掩码机制，所提方法能够在满足容量约束的前提下动态分配缓存资源。

实验结果表明，PPO 策略在训练过程中能够逐步收敛，相比贪心、随机和零缓存策略具有更好的综合性能；Zygote 缓存提供了介于完整缓存和冷启动之间的折中方式，有助于在成本可控的条件下降低 QoS 损失。未来可进一步考虑多边缘节点协同、在线迁移、真实工作负载跟踪和多智能体强化学习等方向，以提升算法在实际云边系统中的适用性。

参考文献

- [1] Satyanarayanan M. The emergence of edge computing[J]. Computer, 2017, 50(1): 30–39.
- [2] Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J/OL]. arXiv:1707.06347, 2017.
- [3] Schulman J, Moritz P, Levine S, et al. High-dimensional continuous control using generalized advantage estimation[J/OL]. arXiv:1506.02438, 2015.
- [4] Wang L, Li M, Zhang Y, et al. Peeking behind the curtains of serverless platforms[C]//USENIX Annual Technical Conference. 2018: 133–146.
- [5] Borodin A, El-Yaniv R. Online Computation and Competitive Analysis[M]. Cambridge: Cambridge University Press, 1998.

附录 A 程序清单

本文实验程序主要包括边缘容器调度环境、PPO 智能体、训练入口和评估脚本。环境模块负责生成请求到达、分类请求和计算奖励；PPO 模块负责动作采样、动作掩码、GAE 估计和策略更新；训练脚本用于批量运行实验并输出图表结果。

附录 B 攻读专业学位硕士期间撰写的论文

- [1] 王宇杰. 边缘计算中面向快速函数启动与低内存开销的在线容器调度方法研究, 课程论文, 2026。

附录 C 攻读专业学位硕士期间申请的专利

- [1] 王宇杰. 一种面向边缘计算的在线容器缓存调度方法, 课程设计材料, 2026。

附录 D 攻读专业学位硕士期间参加的科研项目

- [1] 课程项目，基于 PPO 的边缘容器在线调度仿真实验，2026。

致谢

感谢课程老师在论文写作、科研绘图和 LaTeX 排版方面给予的指导。感谢同学们在实验图整理和模板使用过程中提供的建议。本文内容主要用于课程作业展示，后续仍需在真实边缘计算平台上进一步验证算法性能。